Documentación trabajo de título "forgethreads"

Etapa 3: Mejoras, Pruebas Adicionales y Validación Preliminar

Carlos Riquelme Ávila

30 de octubre de 2026

Resumen

Este documento corresponde a la Etapa 3 del trabajo de título **forgethreads**, una librería en CUDA C++ expuesta a Python para calcular efectos olvidados a gran escala en GPU. Se presentan los ajustes derivados de la revisión de la Etapa 2, la secuencia de optimizaciones aplicadas al kernel max–min (de la versión v1 a la v4 con *tiling* bidimensional), las pruebas de correctitud y de rendimiento realizadas, la validación preliminar sobre matrices de 10.000 nodos y la documentación técnica actualizada. Al cierre de la etapa, el 100 % de las tareas de diseño y el 81 % de las de implementación están completados. Con el kernel v4, forgethreads es en promedio 21,8 veces más rápido que forgeffects y procesa matrices de hasta 37.886 nodos, frente a 1.286.

1. Introducción

En las etapas anteriores se definió el problema (foRgotten [2] y forgeffects no alcanzan las dimensiones que exige la investigación del Dr. Julio Rojas Mora: 2.640 ejecuciones sobre matrices de hasta 10.000 nodos), se diseñó la arquitectura en capas de forgethreads y se implementó una primera versión correcta: el kernel v1, el orquestador iterativo y los bindings PyBind11/DLPack.

La Etapa 3 tiene tres objetivos:

1. **Optimizar** el cómputo partiendo de la línea base medida en la Etapa 2 (kernel v1 limitado por L2), aplicando una mejora a la vez y midiéndola antes y después con Nsight Compute.
2. **Probar** la solución: correctitud de cada variante frente a la referencia CPU, rendimiento frente a forgeffects y comportamiento en los tamaños objetivo.
3. **Validar preliminarmente** el sistema completo en condiciones equivalentes a las de los experimentos de la Etapa 4 (servidor `cuda3`, RTX 4090, $n = 10.000$), y dejar preparada la entrega final.

2. Incorporación de la retroalimentación

No se recibió retroalimentación formal sobre la Etapa 2. Esta sección recoge los compromisos que la propia Etapa 2 dejó abiertos y cómo se resolvieron.

- **Optimización medida, no supuesta.** La Etapa 2 dejó como línea base el perfil del kernel v1 (Tabla 3 de esa etapa). En esta etapa cada mejora se aplicó por separado con el protocolo *perfil antes* → *cambio* → *perfil después* documentado en `mejoras.md`, y se reportan también los resultados que no mejoraron lo esperado (Sección 4).
- **Ajustes de diseño.** El orquestador pasó a trabajar con buffers *padeados* a múltiplos del tamaño de tile y rellenos con un valor negativo (ADR-8), lo que permitió eliminar las guardas de borde del bucle interno de los kernels con *tiling*. La interfaz pública no cambió: los bindings pasaron de invocar `maxmin` (v1) a `maxminv4` sin modificar la firma de `ft.maxmin`, como preveía la arquitectura en capas.

- **Modificaciones técnicas.** La reconstrucción de caminos (T2.1.2b) se integró en el orquestador y el resultado se entrega como la tupla `(paths, values, eta0)` prometida en los requerimientos transversales.
- **Planificación.** El backlog se actualizó con las tareas surgidas durante la etapa (marcadas con $^{[N]}$) y la carta Gantt de la Etapa 4 se rehízo con las desviaciones registradas (Sección 9).
- **Repositorio.** Se adopta Conventional Commits [7] con referencia al ID del backlog para todos los commits nuevos (Sección 8.3).

3. Registro de avance del backlog

La Tabla 1 muestra el estado de cada tarea al cierre de la Etapa 3. Las tareas marcadas con $^{[N]}$ se agregaron durante esta etapa.

Tabla 1: Estado del backlog al cierre de la Etapa 3

ID	Tarea	Tipo	Estado	Etapa
Épica 1 – Límites de forgeffects (OBJ1)				
T1.0.1$^{[N]}$	Diseñar la búsqueda de límites	Diseño	Hecho	1–2
T1.1.1	Script de barrido de dimensión y threshold	Implem.	En progreso	4
T1.1.2	Medición de tiempo con sincronización forzada	Implem.	Hecho	1–2
T1.1.3	Ejecución planificada en Slurm sobre cuda3	Implem.	Hecho	1–2
T1.1.4	Exportación de resultados (media y desviación)	Implem.	Hecho	1–2
T1.1.5	Perfilado de recursos de forgeffects	Validación	En progreso	3–4
Épica 2 – Diseño e implementación de forgethreads (OBJ2)				
T2.0.1$^{[N]}$	Diseñar la arquitectura en capas	Diseño	Hecho	2
T2.1.1	Estrategia de memoria y presupuesto de VRAM	Diseño	Hecho	2
T2.0.3$^{[N]}$	Diseñar el kernel v1	Diseño	Hecho	2
T2.0.4$^{[N]}$	Diseñar la reconstrucción de caminos	Diseño	Hecho	2
T2.0.2$^{[N]}$	Contrato de interfaz y ciclo de vida DLPack	Diseño	Hecho	2
T2.0.5$^{[N]}$	Diseñar las variantes con *tiling* (v2–v4) y el padding	Diseño	Hecho	3
T2.1.2a	Kernel v1 y orquestador iterativo	Implem.	Hecho	2
T2.1.4$^{[N]}$	Verificar el kernel v1 (CPU, compute-sanitizer)	Validación	Hecho	2
T2.3.1	Bindings PyBind11 de `maxmin`	Implem.	Hecho	2
T2.3.3$^{[N]}$	Integración del módulo en CMake	Implem.	Hecho	2
T2.4.1	Importación de tensores vía DLPack	Implem.	Hecho	2
T2.4.3$^{[N]}$	Exportación de buffers vía DLPack	Implem.	Hecho	2
T2.1.2b	Reconstrucción de caminos (host)	Implem.	Hecho	3
T2.5.1	Empaquetado del resultado (caminos, valores, η_0)	Implem.	Hecho	3
T2.2.2	Optimizar los kernels (shared memory, tiling)	Implem.	Hecho	3
T2.1.5$^{[N]}$	Verificar v4 frente a la referencia CPU	Validación	Hecho	3
T2.1.3	Perfilar con Nsight y ajustar a los criterios de OBJ2	Validación	En progreso	3–4
T2.2.1	Benchmark comparativo frente a forgeffects	Implem.	Hecho	3
T2.1.2c$^{[N]}$	Reconstrucción de caminos en GPU (CSR de pivotes)	Implem.	En progreso	4
T2.4.2	Prueba de ausencia de copias host–device	Validación	Pendiente	4
T2.3.2	Empaquetado instalable vía pip	Implem.	Pendiente	4
T2.6.1	Versiones mínimas de CUDA, driver y Python	Document.	Pendiente	4
Épica 3 – Validación de la solución (OBJ3)				

ID	Tarea	Tipo	Estado	Etapa
T3.1.1	Diseñar el pipeline de experimentos de profundidad	Diseño	Hecho	2
T3.2.1	Diseñar el protocolo de toma de tiempo	Diseño	Hecho	2
T3.2.2	Implementar el protocolo de medición	Implem.	Hecho	3
T3.2.3	Calcular y reportar el speedup	Validación	Hecho	3
T3.1.2	Ejecutar y reportar las 2.640 corridas	Validación	Pendiente	4
T3.3.1	Análisis estadístico de los resultados	Validación	Pendiente	4
Documentación				
T4.1[N]	Documento de la Etapa 2	Document.	Hecho	2
T4.3[N]	Documento de la Etapa 3	Document.	Hecho	3
T4.2[N]	Informe final	Document.	Pendiente	4

Tabla 2: Avance por tipo de tarea (Etapa 2 → Etapa 3)

Tipo	Total	Hecho	En progreso	Pendiente	% Etapa 2	% Etapa 3
Diseño	9	9	0	0	100 %	**100 %**
Implementación	16	13	2	1	53 %	**81 %**
Validación	8	3	2	3	14 %	**38 %**
Documentación	4	2	0	2	33 %	**50 %**
Total	37	27	4	6	55 %	**73 %**

El avance de implementación (81 %) supera el 50 % exigido para esta etapa. Cada tarea marcada como *Hecho* tiene respaldo verificable en el repositorio (Sección 5).

4. Mejoras para optimizar el diseño

4.1. Punto de partida

El perfil de Nsight Compute del kernel v1 en la Etapa 2 mostró una ocupancia de 99,16 %, un throughput de cómputo de sólo 5,19 % y un throughput de L2 de 87,09 %: el kernel está **limitado por el acceso a L2**. La causa es estructural: v1 asigna un bloque por celda de salida, de modo que cada fila de C_s y cada columna de A se vuelven a pedir a la jerarquía de memoria una vez por cada celda que las usa. El máximo y el mínimo cuestan una instrucción cada uno; el kernel pasa casi todo su tiempo esperando datos.

La composición max–min tiene la misma estructura de accesos que un GEMM, con $(\text{mín}, \text{máx})$ en lugar de $(\times, +)$. Por eso la estrategia sigue la progresión de Boehm [3] para GEMM: *aumentar la reutilización de cada dato cargado*, primero en shared memory y luego en registros, hasta que el kernel deje de estar limitado por la memoria.

4.2. Variantes implementadas

v1b – columna de A en shared memory. Cada bloque copia una vez la columna $A[b, :, n]$ completa (K elementos, 2 KB para $K = 1.000$) a shared memory mediante una carga cooperativa, y el bucle de reducción lee de ahí. El bucle interno pasa de dos accesos globales por iteración a uno global y uno a shared memory.

v2 – tiling 32×32. Un bloque calcula un tile de 32×32 celdas de salida. En cada paso carga a shared memory un tile de C_s y uno de A, y cada hilo calcula su celda recorriendo los 32 elementos del tile. Cada dato traído de memoria global se reutiliza 32 veces.

v3 – *blocktiling* 1D. Tiles de bloque de 64×64 ($BM = BN = 64$, $BK = 8$) y **8 resultados por hilo** en una columna ($TM = 8$). Cada hilo mantiene en registros los 8 máximos parciales con sus *argmax*; el valor de A leído de shared memory se reutiliza 8 veces desde un registro.

v4 – *blocktiling* 2D. Mismos tiles de bloque, pero cada hilo calcula un sub-tile de $8 \times 8 = 64$ celdas ($TM = TN = 8$), con 64 hilos por bloque. En cada paso del bucle interno el hilo carga 8 valores de C_s y 8 de A desde shared memory a registros y evalúa los 64 pares $\min(\cdot, \cdot)$ con su máximo y *argmax*. La escritura final respeta los límites lógicos $M \times N$ y aplica el umbral diferencial de la Ecuación (2) de la Etapa 2.

Padding. Las variantes v2–v4 suponen dimensiones múltiplo del tile. El orquestador `maxminv4` reserva los buffers con dimensiones *padeadas* y los rellena con un valor negativo (patrón `0xBC` en `half`). Como las incidencias están en $[0, 1]$, un valor negativo es neutro para el máximo y nunca supera el umbral, por lo que el bucle interno no necesita condicionales de borde.

4.3. Análisis de reutilización

La Tabla 3 cuantifica, para una celda de salida y un orden, cuántos accesos requiere cada variante. Es un análisis independiente del hardware que explica por qué cada paso debería reducir la presión sobre L2.

Tabla 3: Accesos por celda de salida y por orden (K = dimensión de la reducción)

Variante	**Lecturas globales**	**Lecturas de shared**	**Reutilización**
v1	$2K$	0	1
v1b	$2K$	K	1
v2 (32×32)	$2K/32$	$2K$	32
v3 (64×64, $TM = 8$)	$2K/64$	$1{,}125\,K$	64
v4 (64×64, 8×8)	$2K/64$	$0{,}25\,K$	64

Reutilización: veces que se usa cada dato traído de memoria global. En v1b la columna se lee de shared memory, lo que quita un acceso global del bucle interno.

Para $n = 10.000$, v1 hace $2 \cdot 10^4$ lecturas globales por celda y v4 hace 313; y v4 reduce además 8 veces las lecturas de shared memory respecto de v2, desplazando la reutilización hacia los registros.

4.4. Comparación antes y después

Mejora v1 → v1b (medida). La Tabla 4 resume el perfil de Nsight Compute antes y después del cambio, con $B = 1$, $n = 1.000$, $\theta = 0{,}5$, en la RTX 4090 de `cuda3`. El conteo de efectos coincide con la referencia CPU en ambos casos (472.326).

Tabla 4: Kernel v1 frente a v1b (Nsight Compute, $B = 1$, $n = 1.000$)

Métrica	**v1**	**v1b**	**Δ**
Duración [ms]	14,58	11,39	−21,9 %
Cómputo (SM) [%]	5,19	10,45	+5,26 pp
Memoria [%]	87,09	90,71	+3,62 pp
L1/TEX [%]	37,87	39,64	+1,77 pp
L2 [%]	87,09	90,71	+3,62 pp
DRAM [%]	0,03	0,04	≈
Ocupancia lograda [%]	99,16	99,55	≈
Registros por hilo	16	26	+10

La duración bajó un 22 %, pero **no por la razón esperada**. La hipótesis era que cachear la columna reduciría la presión sobre L2; el throughput de L2 en cambio *subió*. Cada bloque sigue trayendo la columna completa una vez, así que el volumen total de datos no cambia. La ganancia viene de quitar un acceso global del bucle interno, lo que duplica el throughput de cómputo y hace que el kernel sature L2 durante menos tiempo. El perfil de Nsight Systems de la misma configuración confirma un tiempo de kernel de 11,24 ms.

Este resultado negativo parcial fue decisivo para el diseño: mostró que, mientras cada bloque calcule una sola celda, ninguna caché por bloque reduce el volumen de datos. La reutilización debe ocurrir *entre celdas*, lo que justifica el cambio a *tiling* (v2–v4) en lugar de seguir ajustando v1.

Variantes v1 → v2 → v4 (medida). La Tabla 5 compara el tiempo de extremo a extremo de `ft.maxmin` con cada versión del kernel. Las tres series se midieron con el mismo script (`comparativa/time.py`), las mismas matrices (generador *supercrítico* por bloques, $c = 1$, semillas 101–110), $\theta = 0{,}5$, 3 órdenes y 10 repeticiones por dimensión, en la RTX 4090 de `cuda3`.

Tabla 5: Tiempo de `ft.maxmin` por versión del kernel (ms, media ± desviación estándar)

Dimensión	v1	v2	v4	v1 / v4
150	$0{,}18 \pm 0{,}02$	$0{,}18 \pm 0{,}05$	$0{,}32 \pm 0{,}05$	0,5×
300	$0{,}96 \pm 0{,}05$	$0{,}23 \pm 0{,}03$	$0{,}46 \pm 0{,}04$	2,1×
600	$6{,}33 \pm 0{,}04$	$0{,}75 \pm 0{,}05$	$1{,}02 \pm 0{,}03$	6,2×
700	$9{,}62 \pm 0{,}02$	$1{,}02 \pm 0{,}05$	$1{,}16 \pm 0{,}05$	8,3×
900	$24{,}41 \pm 0{,}09$	$1{,}87 \pm 0{,}05$	$1{,}72 \pm 0{,}04$	14,2×
1.200	$75{,}17 \pm 0{,}10$	$4{,}47 \pm 0{,}25$	$4{,}08 \pm 0{,}07$	**18,4**×

- **v1 crece de forma cúbica**: de 300 a 1.200 (4 veces la dimensión) su tiempo se multiplica por 78, cerca del factor 64 que predice $O(n^3)$ con un kernel limitado por memoria.
- **v2 y v4 crecen mucho más lento**: en el mismo rango se multiplican por 20 y 9. Con estas dimensiones la GPU no se llega a llenar, y el tiempo queda dominado por costos fijos (lanzamientos, reservas y copias) más que por el cómputo.
- **v4 es 18,4 veces más rápido que v1 en** 1.200, y la ventaja crece con la dimensión, como predice la Tabla 3.
- **v2 gana en dimensiones pequeñas** (hasta 700): con 64 hilos por bloque, v4 lanza pocos bloques y deja SM ociosos cuando la matriz es chica. Desde 900 v4 es más rápido (un 9 % en 1.200). Como el objetivo son matrices de hasta 10.000 nodos, se elige v4 como kernel por defecto (ADR-7). **[PENDIENTE: confirmar v2 frente a v4 en $n = 10.000$.]**

Perfil de las variantes. La comparación de esta etapa se hace por tiempo medido con las mismas entradas (Tabla 5). El perfil de Nsight Compute de v4 se realizará en la Etapa 4, junto con la validación final de OBJ2 (ocupancia y throughput $\geq 80\,\%$).

5. Avance de la implementación

5.1. Funcionalidades implementadas

- **Kernels** (`src/kernels/maxmin/`): `kernel_v1.cu` (v1b), `kernel_v2.cu`, `kernel_v3_multC.cu` y `kernel_v4_2D_blocktiling.cu`, todos con la misma firma (valor, *argmax* y contador), lo que permite cambiar de variante sin tocar los bindings.
- **Orquestadores** (`src/algorithms/maxmin{,v2,v3,v4}.cu`): bucle por orden con doble buffer, padding y convergencia anticipada (η_0).

- **Reconstrucción de caminos** (`src/algorithms/assemble_paths.cu`): a partir del *argmax* de cada orden, sin recalcular la composición.
- **Interfaz Python** (`src/bindings.cu`): `ft.maxmin` invoca `maxminv4` y devuelve `(paths, values, eta0)` como arreglos NumPy, un par por orden.
- **Reconstrucción en GPU** (rama `csr-paths-construction`, en progreso): kernels `count_pivots` y `write_pivots` que registran, por orden, los pivotes de cada celda en formato CSR, y un DAG de caminos (`path_dag.cu`) que reconstruye los caminos de m a n sin copiar las matrices densas al host.

5.2. Demostración

```
import numpy as np
import forgethreads as ft

a = np.load("input/CC.npy").astype(np.float16)[None]   # [1, n, n]
paths, values, eta0 = ft.maxmin(a, a, 0.5, 100)
print("profundidad␣de␣detención:", eta0)
for s, (p, v) in enumerate(zip(paths, values), start=1):
    print(f"orden␣{s}:␣{p.shape[0]}␣efectos,␣ancho␣de␣camino␣{p.shape[1]}")
```

El arnés C++ (`src/main.cu`) ejecuta la misma ruta con `maxminv4` y la compara con la referencia CPU (Sección 6).

6. Pruebas iniciales y resultados preliminares

6.1. Correctitud

- **v4 frente a la referencia CPU.** Con $B = 1$, $n = 1.000$ y $\theta = 0{,}5$, el arnés GPU (`src/main.cu`, que invoca `maxminv4`) encuentra 472.326 efectos en el orden 1, exactamente el mismo conteo que la referencia CPU y que v1 (logs `fecuda.848` y `fecuda.850`). El cambio completo de estrategia de cómputo (bloque por celda → tiles 64×64 con padding) preserva el resultado. **[PENDIENTE: repetir la corrida registrando la variante en el log, ya que los logs actuales no la indican.]**
- **Determinismo.** Ejecuciones repetidas sobre la misma entrada producen la misma secuencia de conteos por orden; por ejemplo, para $n = 200$ y `order = 100` se obtiene siempre 2.666, 6.502, 7.655, 4.000, … en las ocho corridas registradas con esa configuración (logs `fecuda.513` a `fecuda.926`).
- **compute-sanitizer.** memcheck, racecheck, initcheck y synccheck: 0 errores para v1 (Etapa 2). **[PENDIENTE: repetir los cuatro modos con v4 vía `scripts/sanitize.sh`.]**

6.2. Rendimiento frente a forgeffects

El benchmark comparativo (`comparativa/time.py`) ejecuta `ft.maxmin` e `iterative_maxmin_cuadrado` de forgeffects sobre **las mismas matrices**: 6 dimensiones, 10 repeticiones por dimensión con semillas distintas, una ejecución previa de calentamiento, $\theta = 0{,}5$ y 3 órdenes, en `cuda3`. Esto corresponde al requerimiento de benchmarking de OBJ3 (6 dimensiones, 10 ejecuciones por dimensión). La Tabla 6 y la Figura 1 muestran los resultados.

Análisis.

- **Criterio de OBJ2 cumplido en el rango medido.** Con v4, forgethreads es en promedio **21,8 veces más rápido** que forgeffects (entre 15,0× y 30,4×), por encima del 10× exigido.

Tabla 6: Tiempo frente a forgeffects (ms, media ± desviación estándar, 10 repeticiones)

Dimensión	forgeffects	ft v4	Speedup v1	Speedup v2	Speedup v4
150	$8{,}50 \pm 1{,}49$	$0{,}32 \pm 0{,}05$	42,8×	44,0×	30,4×
300	$8{,}83 \pm 0{,}62$	$0{,}46 \pm 0{,}04$	8,5×	39,3×	20,7×
600	$15{,}44 \pm 1{,}71$	$1{,}02 \pm 0{,}03$	2,5×	20,7×	15,0×
700	$22{,}52 \pm 5{,}42$	$1{,}16 \pm 0{,}05$	2,2×	19,4×	23,2×
900	$41{,}67 \pm 8{,}42$	$1{,}72 \pm 0{,}04$	1,6×	21,7×	26,2×
1.200	$58{,}94 \pm 3{,}23$	$4{,}08 \pm 0{,}07$	0,8×	12,8×	15,0×
		Speedup medio	9,7×	26,3×	**21,8×**

La columna forgeffects reúne las 30 corridas de forgeffects de las tres campañas. Cada speedup se calcula contra las corridas de forgeffects de su propia campaña.

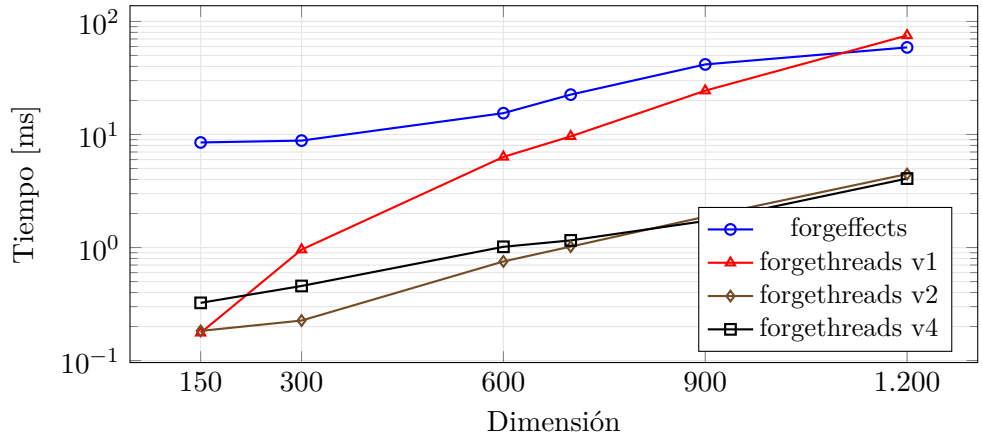


Figura 1: Tiempo de extremo a extremo según la dimensión (escala logarítmica)

El mínimo de cada versión está en 1.200, la dimensión más grande que forgeffects puede procesar.

- **La optimización era necesaria.** Con v1, forgethreads pierde su ventaja al crecer la dimensión y en 1.200 es más lento que forgeffects (0,8×). Sin el *tiling*, OBJ2 no se habría cumplido.
- **Estabilidad.** La desviación estándar de forgethreads es de 0,03 a 0,07 ms desde 600, frente a 1,7–8,4 ms en forgeffects. Los tiempos de forgethreads son predecibles, lo que simplifica estimar la duración de las 2.640 corridas.

Limitación del método. `time.py` mide con `time.time()` alrededor de cada llamada. `ft.maxmin` es síncrona (devuelve arreglos NumPy), pero forgeffects puede devolver tensores de TensorFlow aún en cálculo. **[PENDIENTE: verificar que el resultado de forgeffects se materializa (p. ej. con `.numpy()`) antes de detener el reloj, como exige el requerimiento de medición de OBJ1.]**

6.3. Límites de dimensión y memoria

La búsqueda de límites (`comparativa/memory_bounds.py`: búsqueda exponencial y luego bisección con tolerancia de 10, cada prueba en un subproceso aislado, 2 órdenes) dio los resultados de la Tabla 7.

forgethreads procesa matrices **29 veces más grandes** que forgeffects (en número de elementos, unas 870 veces más). El máximo medido (37.886) coincide con el límite teórico de ≈ 38.700 que la

Tabla 7: Dimensión máxima procesable en la RTX 4090 (24 GB)

Implementación	Máximo	Primer fallo
forgeffects	1.286	1.292: *illegal memory access* en la reducción CUB. Desde 1.500, *OOM* al reservar un tensor $[1, n, n, n]$. Pico de 8,43 GB en 1.282.
forgethreads (versión anterior)	30.650	30.656: un control fijo rechazaba reservas de más de 2 GB por buffer.
forgethreads (actual)	37.886	37.892: la guardia rechaza *argmax* porque superaría el 70 % de la VRAM (17,68 GB).

Etapa 2 calculó a partir del presupuesto $12Bn^2$ bytes, lo que valida el modelo de memoria. Además, la búsqueda confirmó dos comportamientos de forgeffects: falla con un error de índices antes de quedarse sin memoria (Etapa 2) y su consumo pico (8,43 GB para 1.282) es del orden de n^3.

6.4. Prueba con matrices densas

Una medición anterior (`tests/test.py`, $\theta = 0{,}3$, 4 órdenes, matrices densas) mostró el caso opuesto: con 1.000 nodos `ft.maxmin` tardó 497 ms y forgeffects en GPU 394 ms, aunque el kernel sólo toma 11,24 ms por orden (Nsight Systems). Con matrices densas hay cientos de miles de efectos por orden, y la reconstrucción de caminos en el host (copia de C_s, C_{s+1} y *argmax* a la CPU y recorrido en un hilo) domina el tiempo. Esa prueba además construía matrices distintas para cada librería, por eso se reemplazó por `comparativa/time.py`. **[PENDIENTE: confirmar el reparto con un perfil `nsys` de `ft.maxmin` sobre matrices densas.]**

6.5. Ajustes derivados de las pruebas

- **Kernel v4 por defecto** (ADR-7). El benchmark mostró que v1 no cumple OBJ2 y que v4 supera a v2 desde 900.
- **Guardia de VRAM relativa.** La búsqueda de límites mostró que el control fijo de 2 GB por buffer cortaba la dimensión en 30.650, por debajo de lo que la GPU soporta. Se reemplazó por una guardia del 70 % de la VRAM total, con lo que el límite subió a 37.886 (+24 %).
- **Reconstrucción de caminos en GPU** (T2.1.2c$^{[N]}$). La prueba con matrices densas mostró que, con muchos efectos, el costo está fuera del kernel. En la rama `csr-paths-construction` se registran los pivotes de cada orden en formato CSR en la GPU, de modo que sólo se transfieran al host los caminos y no las matrices densas.
- **Modo sin caminos (planificado).** Los experimentos de profundidad sólo necesitan η_0 y los conteos por orden. Se agregará un modo que omita la reconstrucción (ADR-11).

7. Validación preliminar

La validación preliminar se ejecutó en el entorno real de la Etapa 4: servidor `cuda3` (RTX 4090, 24 GB), trabajos Slurm y matrices del tamaño objetivo.

Estimación del costo de la Etapa 4. Con el tiempo medido en $n = 10.000$ con matrices densas y caminos (unos 6,6 s por orden), las corridas con profundidad alta (por ejemplo $\eta_0 = 37$ con $c = 2$) tardarían varios minutos cada una. Sin reconstrucción de caminos, el costo por orden queda determinado por el kernel. **[PENDIENTE: medir el tiempo del kernel v4 por orden en $n = 10.000$ y recalcular la duración estimada de las 2.640 corridas.]**

Tabla 8: Validación preliminar frente a los requerimientos

Requerimiento	**Evidencia**	**Estado**
Soportar $n = 10.000$ (crítico)	Corridas completas con $n = 10.000$ y `order` entre 10 y 25 hasta la convergencia (logs `fecuda.468`–`472`); por ejemplo, 26.644, 26.711, 21.191, ... efectos por orden.	Cumple
Profundidad de detención en $n = 10.000$	$\eta_0 = 3$ con $c = 50$ y $\eta_0 = 37$ con $c = 2$ (`tests/test_10000.csv`, rama de caminos).	Cumple
Prueba de estrés de memoria	La búsqueda de límites llevó la dimensión hasta 38.400; sobre el límite la guardia rechaza la reserva antes de lanzar el kernel y la prueba siguiente continúa sin errores (`comparativa/logs.fecuda.917`).	Cumple
Correctitud frente a CPU	Conteo idéntico (472.326) para v1 y v4.	Cumple
Tiempo en $n = 10.000$	26.532 ms de extremo a extremo para 4 órdenes con $\theta = 0{,}3$ (`output/timing_results2.csv`, una corrida).	Preliminar
Speedup medio $\geq 10\times$ (OBJ2)	21,8× en promedio con v4, mínimo 15,0×, mismas entradas, 6 dimensiones y 10 repeticiones (Tabla 6).	Cumple (hasta 1.200)
Superar el límite de forgeffects	37.886 frente a 1.286 nodos (Tabla 7).	Cumple
Ocupancia y throughput $\geq 80\,\%$ (OBJ2)	Ocupancia 99,55 % y memoria 90,71 % en v1b; perfil de v4 en la Etapa 4.	Parcial

8. Documentación del proyecto

8.1. Decisiones técnicas (ADR)

Se mantienen ADR-1 a ADR-6 de la Etapa 2 y se agregan:

Tabla 9: Nuevas decisiones de la Etapa 3

ID	**Decisión**	**Justificación y consecuencia**
ADR-7	*Blocktiling* 2D (v4) como kernel por defecto	Máxima reutilización por dato global y en registros (Tabla 3). Los bindings cambian a `maxminv4` sin alterar la API. A cambio, el kernel es más complejo y requiere dimensiones padeadas.
ADR-8	Padding a múltiplos del tile con relleno negativo	Elimina las guardas de borde del bucle interno; el valor negativo es neutro para el máximo porque las incidencias están en $[0, 1]$. Cuesta hasta 63 filas y columnas extra de memoria (48 para $n = 10.000$, menos del 1 %).
ADR-9	Mantener v1–v3 en el repositorio	Sirven como referencia de correctitud y como evidencia de la progresión de optimización; comparten firma con v4, por lo que pueden compararse en el mismo arnés.

ADR-10	Reconstrucción de caminos en GPU con CSR de pivotes	Las pruebas mostraron que el costo de extremo a extremo está fuera del kernel. Registrar pivotes en CSR en la GPU evita copiar matrices densas al host en cada orden.
ADR-11	Modo sin caminos para los experimentos	Los experimentos de profundidad sólo necesitan η_0 y los conteos por orden; omitir los caminos reduce el tiempo de las 2.640 corridas. Se implementa en la Etapa 4.

8.2. Desviaciones del plan

- **Speedup en $n = 10.000$ a la Etapa 4.** forgeffects no supera 1.286 nodos, así que el speedup sólo puede medirse hasta 1.200. En 10.000 se reportará el tiempo absoluto de forgethreads.
- **Nueva tarea T2.1.2c$^{[N]}$.** No estaba planificada; surgió de la prueba con matrices densas (Sección 6).
- **Script de medición reemplazado.** `tests/test.py` se sustituyó por `comparativa/time.py` al detectar que construía matrices distintas para cada librería.
- **Barrido de threshold (T1.1.1) a la Etapa 4.** Se priorizó la optimización, que es el camino crítico para las 2.640 corridas. Del perfil de forgeffects (T1.1.5) ya están medidos el límite de dimensión y el pico de memoria; falta el perfil Nsight.
- **Perfil Nsight de v2–v4 (T2.1.3) a la Etapa 4.** La mejora se demostró con tiempos medidos sobre las mismas entradas; el perfil de v4 se hará junto con la validación final de los criterios de ocupancia y throughput de OBJ2.
- **Prueba de ausencia de copias (T2.4.2) a la Etapa 4.** Hoy la salida de caminos pasa necesariamente por el host; la prueba sólo tiene sentido una vez integrada la reconstrucción en GPU.

8.3. Repositorio git

El proyecto se versiona en `https://github.com/criquelme2003/feCUDA`. El trabajo de esta etapa se encuentra en las ramas `gpu_paths` (kernels v2–v4 y orquestadores) y `csr-paths-construction` (reconstrucción en GPU). Commits relevantes: `cf44ae8` (kernel v2), `17f70a4` (kernels v3 y v4), `fa645cf` (optimización de los kernels max–min), `322cef9` (corrección de caminos fallidos) y `96c2da7` (registro CSR por orden y reconstrucción de caminos de m a n), `0aad706` (benchmark de límites y tiempo) y `a0b0dd8` (benchmark de tiempo). Los datos crudos de las Tablas 5, 6 y 7 están en `comparativa/` (`time_results*.csv` y `logs.*`).

Desde esta etapa los commits siguen Conventional Commits [7]:

```
perf(kernels): add 2D blocktiling maxmin kernel (v4)

Each thread computes an 8x8 sub-tile so values loaded from shared
memory are reused 8 times from registers. Matches CPU reference
(472326 effects, n=1000, thr=0.5).

Refs: T2.2.2
```

9. Planificación: Etapa 4

La Etapa 4 empieza por cerrar lo que la Etapa 3 dejó abierto: perfilar v4 con Nsight, integrar la reconstrucción de caminos en GPU y confirmar que la elección de v4 se mantiene en $n = 10.000$. Con eso se lanzan las 2.640 corridas en Slurm, en paralelo con el empaquetado. El análisis estadístico y el informe final cierran el trabajo antes del 4 de diciembre.

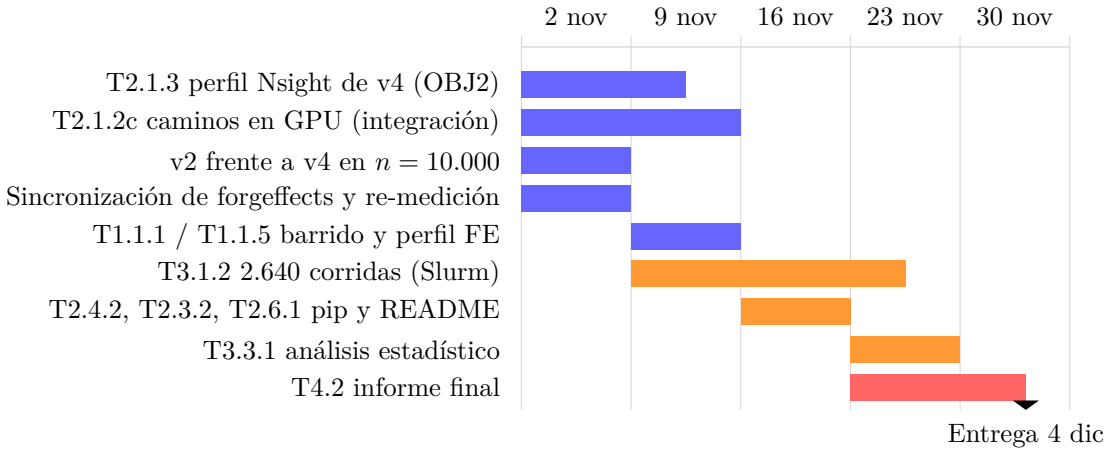


Figura 2: Carta Gantt de la Etapa 4

Referencias

[1] A. Kaufmann y J. Gil Aluja, *Modelos para la investigación de efectos olvidados*. Vigo: Milladoiro, 1988.

[2] E. Mardones-Arias y J. Rojas-Mora, *foRgotten: paquete de GNU R para el análisis de efectos olvidados*, 2022.

[3] S. Boehm, «How to Optimize a CUDA Matmul Kernel for cuBLAS-like Performance: a Worklog», 2022. [En línea]. Disponible: `https://siboehm.com/articles/22/CUDA-MMM`

[4] NVIDIA Corporation, *CUDA C++ Best Practices Guide*, v12.5, 2024.

[5] NVIDIA Corporation, *Nsight Compute Documentation*, 2024. [En línea]. Disponible: `https://docs.nvidia.com/nsight-compute/`

[6] M. Nygard, «Documenting Architecture Decisions», 2011. [En línea]. Disponible: `https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions`

[7] «Conventional Commits 1.0.0». [En línea]. Disponible: `https://www.conventionalcommits.org`